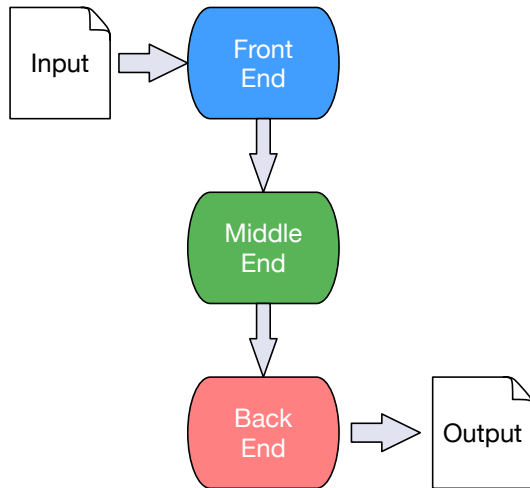


# Compiler Construction

～ Middle End ～

# General layout – Compiler structure



# Goal of middle end

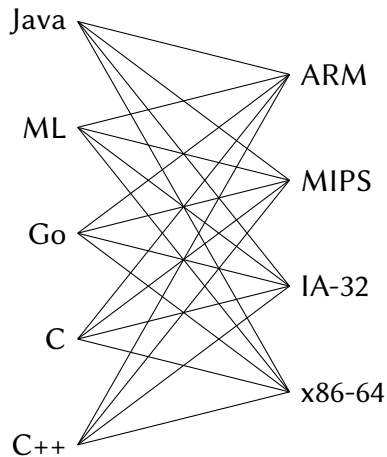
- Generic optimisations
- Prepare and simplify backend !
  - ▶ No nested sequences in ASM
  - ▶ No expressions in ASM
  - ▶ No two-way conditional jumps in ASM
  - ▶ ASM has calling convention

# Goal of middle end

- Generic optimisations
- Prepare and simplify backend !
  - ▶ No nested sequences in ASM
  - ▶ No expressions in ASM
  - ▶ No two-way conditional jumps in ASM
  - ▶ ASM has calling convention

Do we require an intermediate representation?

# Retargetable Compilers (1/2)



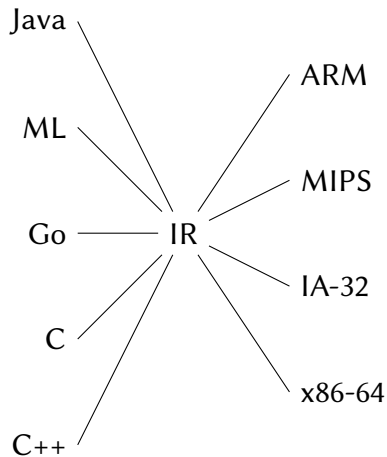
# Goal of middle end

An intermediate representation is not *required*.

**BUT**

.. preferable for optimisations and retargetable compilers.

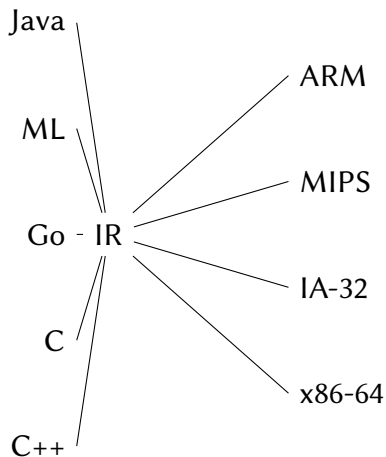
## Retargetable Compilers (2/2)



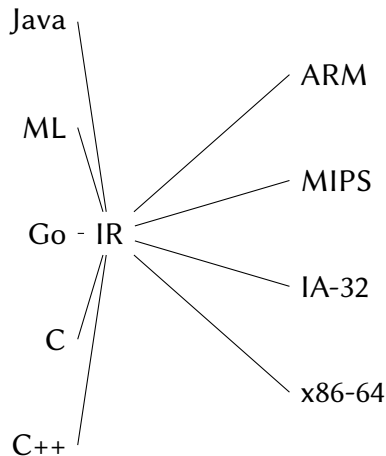
# What abstraction?

What is the best abstraction level for the intermediate representation?

## Strategy 1: close to front end

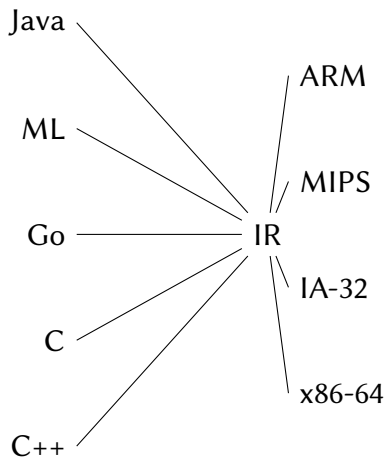


## Strategy 1: close to front end

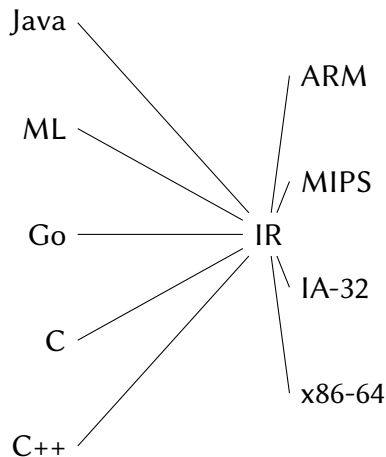


- Easy translation from source language
- Very hard translation from IR to ASM

## Strategy 2: close to back end

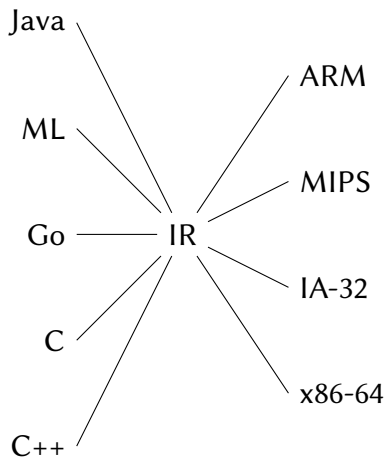


## Strategy 2: close to back end

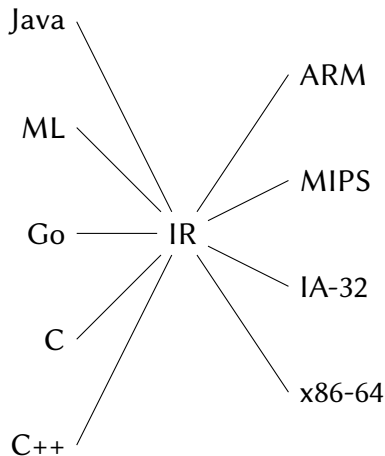


- Very hard translation from source language
- Easy translation from IR to ASM

## Strategy 3: barycenter of sources and targets

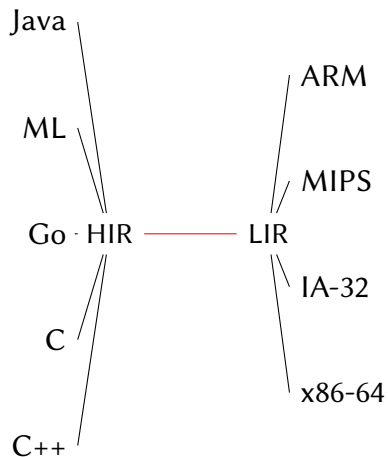


## Strategy 3: barycenter of sources and targets

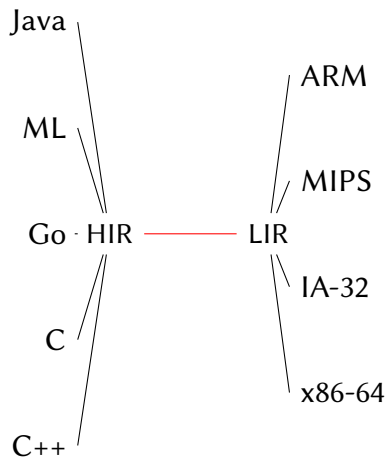


- Hard to find the barycenter of languages
- Not so easy translation from source to IR and from IR to ASM

## Strategy 4: Two IRs

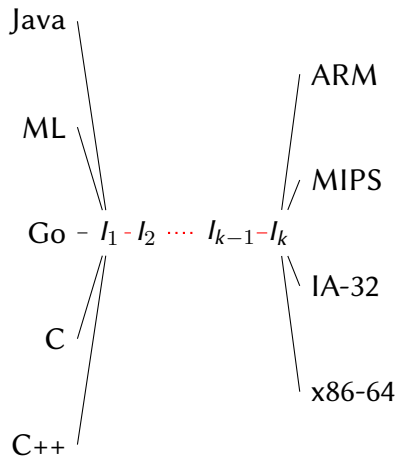


## Strategy 4: Two IRs

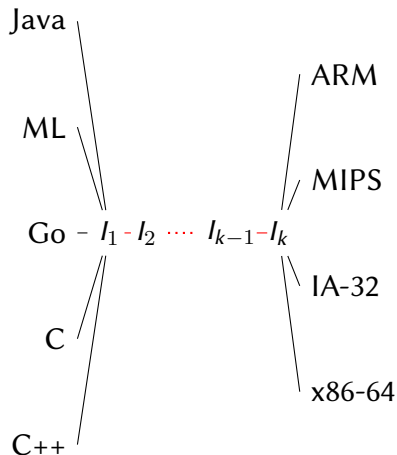


- Easy translation from source to HIR
- Hard translation from HIR to LIR
- Easy translation from LIR to ASM

## Strategy 5: Many IRs



## Strategy 5: Many IRs



- Easy translation from source to  $I_1$
- Easy translation from  $I_j$  to  $I_{j+1}$ 
  - ↪ but hard to define multiple language and lots of intermediate translations
- Easy translation from  $I_k$  to ASM

# Solution

Define  $l_1, l_2, \dots, l_{k-1}, l_k$  as the same language!

$l_{k-1}$  is sugar over  $l_k$

...

$l_2$  is sugar over  $l_3$

$l_1$  is sugar over  $l_2$

# Solution

Define  $l_1, l_2, \dots, l_{k-1}, l_k$  as the same language!

$l_{k-1}$  is sugar over  $l_k$

...

$l_2$  is sugar over  $l_3$

$l_1$  is sugar over  $l_2$

⇒ Translation from HIR to LIR will only be a **form** of unsugarring

# Format? Representation? Language?

## *Intermediate representation:*

- a faithful model of the source program
- “written” in an abstract language, the *intermediate language*
- may have an external syntax
- may be interpreted/compiled (oVM, byte code)
- may have different levels (GCC’s Tree is very much like C).

# What Language Flavor?

- **Imperative?**

- ▶ **Stack Based?** (Java Byte-code)
- ▶ **Register Based?** (GCC's RTL,  $\text{tC}$ 's Tree)

- **Functional?**

Most functional languages are compiled into a lower level language, eventually a simplified version of  $\lambda$ -calculus.

- **Other?**

# What Level?

A whole range of expressivities, typically aiming at making some optimizations easier:

- Keep array expressions?

**Yes:** adequate for dependency analysis and related optimizations,

**No:** Good for constant folding, strength reduction, loop invariant code motion, etc.

- Keep loop constructs?

What level of machine independence?

- Explicit register names?

# Designing an Intermediate Representation

“ *Intermediate-language design is largely an art, not a science.*

—  
*Muchnick, 1997*

# Different Levels [Muchnick, 97]

```
float a[20][10];  
...  
print(a[i][j+2]);
```

## Different Levels [Muchnick, 97]

```
float a[20][10];  
...  
print(a[i][j+2]);
```

```
t1 <- a[i,j+2]  
// funcall on t1
```

## Different Levels [Muchnick, 97]

```
float a[20][10];  
...  
print(a[i][j+2]);
```

```
t1 <- a[i,j+2]  
// funccall on t1
```

```
t1 <- j + 2  
t2 <- i * 20  
t3 <- t1 + t2  
t4 <- 4 * t3  
t5 <- addr a  
t6 <- t5 + t4  
t7 <- *t6  
// funccall on t7
```

# Different Levels [Muchnick, 97]

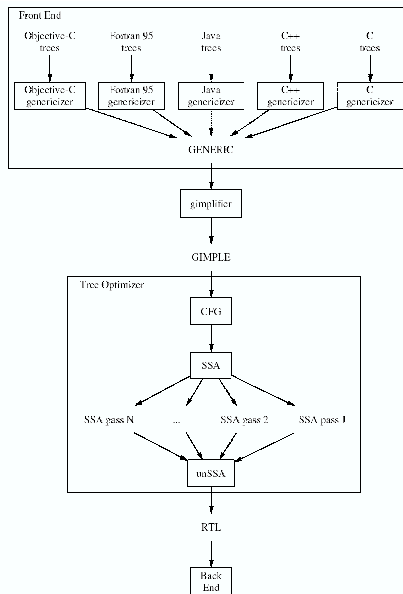
```
float a[20][10];  
...  
print(a[i][j+2]);
```

```
t1 <- a[i,j+2]  
// funcall on t1
```

```
t1 <- j + 2  
t2 <- i * 20  
t3 <- t1 + t2  
t4 <- 4 * t3  
t5 <- addr a  
t6 <- t5 + t4  
t7 <- *t6  
// funcall on t7
```

```
r1 <- [fp - 4]  
r2 <- r1 + 2  
r3 <- [fp - 8]  
r4 <- r3 * 20  
r5 <- r4 + r2  
r6 <- 4 * r5  
r7 <- fp - 216  
f1 <- [r7 + r6]  
// funcall on f1
```

# Different Levels: The GCC Structure



## Stack Based: Java Byte-Code [Edwards, 03]

```
class Gcd {  
    static public int gcd(int a, int b) {  
        while (a != b) {  
            if (a > b)  
                a -= b;  
            else  
                b -= a;  
        }  
        return a;  
    }  
  
    static public int main(String[] arg) {  
        return gcd(12, 34);  
    }  
}
```

# Stack Based: Java Byte-Code

```
% gcj-3.3 -c gcd.java
% jcf-dump-3.3 -c gcd
...
Method name: "gcd" public static
Signature: 5=(int,int)int
Attribute "Code", length:66,
max_stack:2, max_locals:2,
code_length:26
  0: iload_0
  1: iload_1
  2: if_icmpeq 24
  5: iload_0
  6: iload_1
  7: if_icmple 17
10: iload_0
11: iload_1
12: isub
```

```
13: istore_0
14: goto 21
17: iload_1
18: iload_0
19: isub
20: istore_1
21: goto 0
24: iload_0
25: ireturn
Attribute "LineNumberTable",
      length:22, count: 5
  line: 5 at pc: 0
  line: 7 at pc: 5
  line: 8 at pc: 10
  line: 10 at pc: 17
  line: 12 at pc: 24
...
```

# Stack Based

## Advantages

- Simple translation of expressions
- Trivial to interpret
- No pressure on registers
- Often compact

# Stack Based

## Advantages

- Simple translation of expressions
- Trivial to interpret
- No pressure on registers
- Often compact

## Disadvantages

- Does not translate well with today's architectures
- Hard to analyze
- Hard to optimize

# Register Based: What Structure?

How is the structure coded?

**Addresses** Expressions and instructions have names, or (absolute) addresses. (Stack based is a bit like a relative address).

# Register Based: What Structure?

How is the structure coded?

**Addresses** Expressions and instructions have names, or (absolute) addresses. (Stack based is a bit like a relative address).

- 2 address instructions? (**triples**)

# Register Based: What Structure?

How is the structure coded?

**Addresses** Expressions and instructions have names, or (absolute) addresses. (Stack based is a bit like a relative address).

- 2 address instructions? (**triples**)
- 3 address instructions? (**quadruples**)

# Register Based: What Structure?

How is the structure coded?

**Addresses** Expressions and instructions have names, or (absolute) addresses. (Stack based is a bit like a relative address).

- 2 address instructions? (**triples**)
- 3 address instructions? (**quadruples**)

**Tree** Expressions and instructions are unnamed, related to each other as nodes of trees

# Register Based: What Structure?

How is the structure coded?

**Addresses** Expressions and instructions have names, or (absolute) addresses. (Stack based is a bit like a relative address).

- 2 address instructions? (**triples**)
- 3 address instructions? (**quadruples**)

**Tree** Expressions and instructions are unnamed, related to each other as nodes of trees

**DAG** Compact, good for local value numbering, but that's all.

# Quadruples vs. Triples

```
L1:  i  <- i + 1  
  
      t1 <- i + 1  
      t2 <- p + 4  
      t3 <- *t2  
      p  <- t2  
      t4 <- t1 < 10  
      *r <- t3  
      if t4 goto L1
```

```
(1)  i + 1  
(2)  i sto (1)  
(3)  i + 1  
(4)  p + 4  
(5)  *(4)  
(6)  p sto (4)  
(7)  (3) < 10  
(8)  *r sto (5)  
(9)  if (7), (1)
```

# Register Based: GCC's RTL

```
int gcd(int a, int b) {  
    while (a != b) {  
        if (a > b)  
            a -= b;  
        else  
            b -= a;  
    }  
    return a;  
}
```

## Register Based: GCC's RTL (1/2)

```
gcc -da -c gcd.c
```

Generates many files (one per pass).

gcd.c.338r.dfinit (excerpt):

```
;; Function gcd (gcd, funcdef_no=0, decl_uid=1980,  
  cgraph_uid=1, symbol_order=0)  
  
(note 1 0 5 NOTE_INSN_DELETED)  
(note 5 1 44 2 [bb 2] NOTE_INSN_BASIC_BLOCK)  
(insn/f 44 5 45 2 (set (mem:DI (pre_dec:DI (reg/f:DI 7 sp))  
  [0 S8 A8])  
  (reg/f:DI 6 bp)) "gcd.c":1:23 54 { *pushdi2_rex64}  
(nil))
```

## Register Based: GCC's RTL (2/2)

```
(note 23 22 25 6 [bb 6] NOTE_INSN_BASIC_BLOCK)
(insn 25 23 26 6 (set (reg:SI 0 ax [87])
  (mem/c:SI (plus:DI (reg/f:DI 6 bp)
    (const_int -4 [0xfffffffffffffffffc]))) [1 a+0 S4 A32])) "
    gcd.c":2:14 81 {*movsi_internal}
  (nil))
(insn 26 25 27 6 (set (reg:CCZ 17 flags)
  (compare:CCZ (reg:SI 0 ax [87])
    (mem/c:SI (plus:DI (reg/f:DI 6 bp)
      (const_int -8 [0xfffffffffffffffff8]))) [1 b+0 S4 A32
    ]))) "gcd.c":2:14 11 {*cmpsi_1}
  (nil))
(jump_insn 27 26 28 6 (set (pc)
  (if_then_else (ne (reg:CCZ 17 flags)
    (const_int 0 [0]))
    (label_ref 24)
    (pc))) "gcd.c":2:14 891 {*jcc}
  (nil)
-> 24)
```

# Register Based

## Advantages

- Suits today's architectures
- Clearer data flow

# Register Based

## Advantages

- Suits today's architectures
- Clearer data flow

## Disadvantages

- Harder to synthesize
- Less compact
- Harder to interpret

# Tree [Appel, 98]

A simple intermediate language:

- Tree structure
- Expression and Instructions
- "real" function calls

More flexible than assembly

- Unbounded number of registers (temporaries)
- Two way conditional jump
- No conventions

# Tree: Grammar

```
<Exp> ::= "const" int
        | "constF" float
        | "name" <Label>
        | "temp" <Temp>
        | "binop" <Op> <Exp> <Exp>
        | "mem" <Exp>
        | "call" <Exp> [ {<Typ> <Exp>} ] "call end"
        | "callF" <Exp> [ {<Typ> <Exp>} ] "call end"
        | "eseq" <Stm> <Exp>
```

```
<Op> ::= "add" | "sub" | "mul" | "div" | "mod"
        | "addF" | "subF" | "mulF" | "divF"
```

```
<Rop> ::= "eq" | "ne" | "lt" | "gt" | "le" | "ge"
         | "eqF" | "neF" | "ltF" | "gtF" | "leF" | "geF"
```

# Tree: Grammar

```
<Stm> ::= "move" <Exp> <Exp>
        | "sxp" <Exp>
        | "jump" <Exp> [ {<Label>} ]
        | "cjump" <Rop> <Exp> <Exp> "name" <Label> "name" <Label>
        | "seq" [ {<Stm>} ] "seq end"
        | "label" <Label>
```

sxp is OCaml's ignore

# What temporaries are?

A temp is an *abstract register*:

- unlimited supply
- no hardware meaning
- later mapped to real registers or stack slots

Kinds of temporaries

User variables  $t_1, t_2, \dots$

Created on first assignment  
(newTemp() in assignments)

Return values  $rv, fv$

Convention: integer vs floating-point return

Parameters  $i_0, i_1, \dots \quad fi_0, fi_1, \dots$

Bound at function entry according to argument order and type

# What temporaries are not?

A temp is **not a memory location** and **not a register**.

It represents a *named value* produced by an expression:

- no address
- no aliasing
- no fixed storage

Temporaries encode **data flow** (who uses whose result), while **storage decisions are deferred** to later phases.

# Tree Samples - Arithmetics

1 + 2 \* 3

```
/* == High Level Intermediate representation. == */  
# Routine: main  
label Main  
# Prologue  
# Body  
sxp  
    binop add  
        const 1  
        binop mul  
            const 2  
            const 3  
# Epilogue  
label end
```

# Tree Samples - Blocks

```
let x = 22 in x + 20
```

```
/* == High Level Intermediate representation. == */  
# Routine: main  
label Main  
# Prologue  
# Body  
sxp  
  eseq  
    move  
      temp t1  
      const 22  
    binop add  
      temp t1  
      const 20  
# Epilogue  
label end
```

# Tree Samples - Function calls / Strings

```
print("coucou")
```

```
/* == High Level Intermediate representation. == */  
label 10 "coucou"  
# Routine: _main  
label main  
# Prologue  
# Body  
seq  
  sxp  
    call  
      name print  
      name 10  
    call end  
seq end  
# Epilogue  
label end
```

# Tree Samples - If Then Else

```
if true then print_int (1)
```

```
/* == High Level Intermediate representation. == */  
# Routine: Main Program  
label Main  
# Prologue  
# Body  
seq  
  cjump ne, const 1, const 0, name 11, name 12  
  label 11  
  sxp call name print_int, const 1  
  jump name 13  
  label 12  
  sxp const 0  
  label 13  
seq end  
# Epilogue  
label end
```

# Tree Samples - While

```
while true do print_int (1)
```

```
/* == High Level Intermediate representation. == */  
# Routine: Main Program  
label Main  
# Prologue  
# Body  
seq  
  label 11  
  cjump ne const 1 const 0 name 12 name 10  
  label 12  
  sxp  
    call  
      name print_int  
      const 1  
    call end  
  jump name 11  
  label 10  
seq end  
# Epilogue
```

# Tree Samples - Function Definitions

```
let function gcd(a: int, b: int) : int = (  
    while a <> b do  
        if a > b then  
            a := a - b  
        else  
            b := b - a;  
        a  
    )  
in  
    print_int(gcd(42, 51))  
end
```

# Tree samples (1/2)

```
# Routine: gcd
label 10
move temp rv
    eseq
    seq
        label 12 /* While entry point */
        cjump ne i0 i1 name 13 name 11
        label 13
        seq
            cjump gt i0 i1 name 14 name 15
            label 14
            move i0 binop sub i0 i1
            jump name 16
            label 15
            move i1 binop sub i1 i0
            label 16
        seq end
        jump name 12 /* go back to while's entry point */
    label 11
seq end
i0
```

## Tree samples (2/2)

```
# Routine: _main
label main
seq
  sxp
    call
      name print_int
      call name 10 const 42 const 51
      call end
    call end
  sxp
    const 0
seq end
label end
```

# Trees + Frames

So far, our intermediate trees assumed:

- Infinite temporaries
- Direct access to parameters and locals
- No explicit calling convention

In a real machine-oriented IR, this is no longer sufficient. Each function must:

- Establish its own *activation record* (stack frame)
- Save and restore caller/callee state
- Access parameters and local variables via the frame pointer

# Activation Frames — What?

An *activation frame* (or stack frame) is the runtime data structure associated with a single function invocation. It (generally) contains:

- The function's parameters
- Local variables
- Temporaries that must survive across expressions
- Saved machine state (e.g., return address, old frame pointer)

Each **call** creates a new activation frame, which is:

- Allocated on entry to the function
- Accessed via a *frame pointer* (fp)
- Deallocated when the function returns

# Activation Frames — Why?

Earlier intermediate trees were sufficient for expressing:

- Control flow
- Expression evaluation

But are no longer sufficient when targeting execution on real machines:

- Temporaries cannot live indefinitely
- Parameters and locals must have concrete storage locations
- Recursive calls require distinct instances of the same variables

# Tree samples (1/4) - Frame Handling

```
/* == High Level Intermediate representation. == */  
# Routine: gcd  
label 10  
# Prologue  
move temp t0 temp fp  
move temp fp temp sp  
move  
    temp sp  
    binop sub temp sp const 12  
move  
mem temp fp  
    temp i0  
move  
    mem binop add temp fp const -4  
    temp i1  
move  
    mem binop add temp fp const -8  
    temp i2
```

## Tree samples (2/4) - Frame Handling

```
# Body
move temp rv
  eseq
  seq
    label 12 /* While entry point */
    cjump ne mem binop add temp fp const -4
           mem binop add temp fp const -8
           name 13 name 11
    label 13
    seq
      cjump gt mem binop add temp fp const -4
            mem binop add temp fp const -8
            name 14 name 15
      label 14
      move mem binop add temp fp const -4
            binop sub mem binop add temp fp const -4
                  mem binop add temp fp const -8
      jump name 16
```

## Tree samples (3/4) - Frame Handling

```
label 15
  move mem binop add temp fp const -8
      binop sub mem binop add temp fp const -8
          mem binop add temp fp const -4

label 16
seq end
jump name 12 /* go back to while's entry point */
label 11
seq end
mem binop add temp fp const -4

# Epilogue
move temp sp temp fp
move temp fp temp t0
label end
```

## Tree samples (4/4) - Frame Handling

```
# Routine: _main
label main
# Prologue
# Body
seq
  sxp
    call
      name print_int
      call name 10 temp fp const 42 const 51
      call end
    call end
  sxp
    const 0
  seq end
# Epilogue
label end
```